

On-Policy Temporal-Difference Control: n -Step SARSA

Reinforcement Learning

Sai Sampath Kedari

sampath@umich.edu

Contents

1	Setup and What Is New	2
2	Unrolling the Bellman Equation	2
2.1	The One-Step Equation and Its Bootstrap	2
2.2	Unrolling Once: The Two-Step Equation	2
2.3	Unrolling n Times: The n -Step Equation	3
3	From the n-Step Equation to the Update	3
3.1	Sampling	3
3.2	The Update	4
3.3	Bias and Variance	4
3.4	Backup Diagram	5
4	Closing Remarks	5
4.1	The Lag in Updating	5
4.2	Convergence	6
4.3	Summary	6

1 Setup and What Is New

We continue with the infinite-horizon discounted MDP of the SARSA(0) report: finite state space $\mathcal{S}$, finite action sets $\mathcal{A}(s)$, joint transition kernel $p(s', r \mid s, a)$, discount $\gamma \in [0, 1)$, and stationary policies $\pi(a \mid s)$.

The SARSA(0) report converted the one-step Bellman expectation equation into a sampled update through a fixed procedure: write q_π as a conditional expectation, replace each expectation by one sample drawn from the trajectory, bootstrap the unknown q_π with the current estimate Q , and update Q as a running mean of the resulting target. The improvement step is ε -greedy and is woven in at every step. We take all of that as given.

This report changes exactly one thing: *how far we unroll the Bellman equation before bootstrapping*. SARSA(0) bootstraps after one real step. n -step SARSA takes n real steps before bootstrapping. The conversion from operator to sample is otherwise identical; the only new derivation is the unrolling itself, which is the subject of Section 2. Everything downstream (sampling, running mean, GPI, on-policy interpretation) is cited from the SARSA(0) report.

2 Unrolling the Bellman Equation

2.1 The One-Step Equation and Its Bootstrap

The compact form of the one-step Bellman expectation equation, derived in the SARSA(0) report, is

$$q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a].$$

The right-hand side contains the term $q_\pi(S_{t+1}, A_{t+1})$: the action-value at the next state-action pair, used as the bootstrap. The key observation is that this term is itself an action-value, so it satisfies the same Bellman equation, one step later in time.

2.2 Unrolling Once: The Two-Step Equation

Write the Bellman equation at the next pair (S_{t+1}, A_{t+1}) :

$$q_\pi(S_{t+1}, A_{t+1}) = \mathbb{E}_\pi[R_{t+2} + \gamma q_\pi(S_{t+2}, A_{t+2}) \mid S_{t+1}, A_{t+1}].$$

This is an identity in the random variables (S_{t+1}, A_{t+1}) : whatever values they take, the action-value at that pair equals the expected immediate reward plus the discounted bootstrap, conditioned on that pair.

Substitute this expression for $q_\pi(S_{t+1}, A_{t+1})$ into the one-step equation:

$$q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma \mathbb{E}_\pi[R_{t+2} + \gamma q_\pi(S_{t+2}, A_{t+2}) \mid S_{t+1}, A_{t+1}] \mid S_t = s, A_t = a].$$

We collapse the nested expectation using the tower property. The inner expectation is conditioned on (S_{t+1}, A_{t+1}) ; the outer is conditioned on $(S_t, A_t) = (s, a)$. Since (S_{t+1}, A_{t+1}) is later along the same policy-environment trajectory than (S_t, A_t) , conditioning on it is a refinement of conditioning on (s, a) . The tower property

$$\mathbb{E}_\pi[\mathbb{E}_\pi[Y \mid S_{t+1}, A_{t+1}] \mid S_t = s, A_t = a] = \mathbb{E}_\pi[Y \mid S_t = s, A_t = a]$$

applied to $Y = R_{t+2} + \gamma q_\pi(S_{t+2}, A_{t+2})$ removes the inner conditioning. Using linearity to keep R_{t+1} outside, we obtain the **two-step Bellman equation**:

$$q_\pi(s, a) = \mathbb{E}_\pi [R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, A_{t+2}) \mid S_t = s, A_t = a].$$

The bootstrap has moved two steps out, and a second real reward R_{t+2} has entered the sum. Nothing else changed.

Unrolling is one move: the bootstrap term $q_\pi(S_{t+1}, A_{t+1})$ satisfies the same Bellman equation, so we substitute that equation in for it. Each substitution trades one bootstrap for one more real reward plus a bootstrap one step deeper. The tower property is what lets the nested conditional expectations collapse into a single expectation conditioned on the starting pair.

2.3 Unrolling n Times: The n -Step Equation

Repeating the substitution n times pushes the bootstrap to step n . At each repetition we replace the current deepest bootstrap term $q_\pi(S_{t+k}, A_{t+k})$ with its own Bellman equation, collapse the new nested expectation by the tower property, and pick up one more real reward R_{t+k+1} with discount γ^k . After n repetitions:

$$q_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{i=1}^n \gamma^{i-1} R_{t+i} + \gamma^n q_\pi(S_{t+n}, A_{t+n}) \mid S_t = s, A_t = a \right].$$

This is the **n -step Bellman equation**. The first n rewards are real and discounted by $\gamma^0, \dots, \gamma^{n-1}$; the bootstrap $q_\pi(S_{t+n}, A_{t+n})$ is discounted by γ^n and stands in for everything beyond step n .

Two limits anchor the family. Setting $n = 1$ recovers the one-step Bellman equation of SARSA(0). Letting $n \rightarrow \infty$ removes the bootstrap entirely ($\gamma^n \rightarrow 0$ and the sum becomes the full return), recovering the definition $q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid s, a]$, which is the Monte Carlo case.

3 From the n -Step Equation to the Update

3.1 Sampling

The n -step Bellman equation is a conditional expectation in exactly the form the statistical move requires. We apply the move from the SARSA(0) report without re-deriving it: draw one realization of each random variable along the trajectory, and bootstrap the unknown q_π with the current estimate Q .

Starting from $(S_t, A_t) = (s, a)$, run the policy for n steps in the environment, observing

$$R_{t+1}, S_{t+1}, A_{t+1}, R_{t+2}, S_{t+2}, A_{t+2}, \dots, R_{t+n}, S_{t+n}, A_{t+n},$$

with each $(S_{t+i}, R_{t+i}) \sim p(\cdot, \cdot \mid S_{t+i-1}, A_{t+i-1})$ and each $A_{t+i} \sim \pi(\cdot \mid S_{t+i})$. Evaluating the integrand on these realizations and bootstrapping with Q gives the **n -step target**:

$$q_t^{(n)} = \sum_{i=1}^n \gamma^{i-1} R_{t+i} + \gamma^n Q(S_{t+n}, A_{t+n}).$$

3.2 The Update

The running-mean update steps $Q(S_t, A_t)$ toward the n -step target:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(q_t^{(n)} - Q(S_t, A_t)).$$

Setting $n = 1$ gives $q_t^{(1)} = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$, the SARSA(0) update.

3.3 Bias and Variance

The depth n controls a bias-variance tradeoff, and this is the entire reason to use $n > 1$.

Bias comes from the bootstrap. The target $q_t^{(n)}$ uses $Q(S_{t+n}, A_{t+n})$ in place of the unknown $q_\pi(S_{t+n}, A_{t+n})$, and the two are equal only at convergence. The bootstrap is discounted by γ^n , so the larger n is, the less weight the (biased) bootstrap carries, and the smaller the bias. At $n \rightarrow \infty$ there is no bootstrap and the target G_t is unbiased for q_π .

Variance comes from the sampled rewards. The target $q_t^{(n)}$ sums n random rewards along one random trajectory, so its variance grows with n . At $n = 1$ only a single reward and a single bootstrap contribute, giving the lowest variance.

n interpolates between SARSA(0) and Monte Carlo control. Small n : low variance, high bootstrap bias. Large n : high variance, low bootstrap bias. Intermediate n often learns faster than either extreme, because it balances the two. This tradeoff is purely a consequence of how many real steps replace the bootstrap; it has nothing to do with the structure of the algorithm, which is identical to SARSA(0) at every depth.

3.4 Backup Diagram

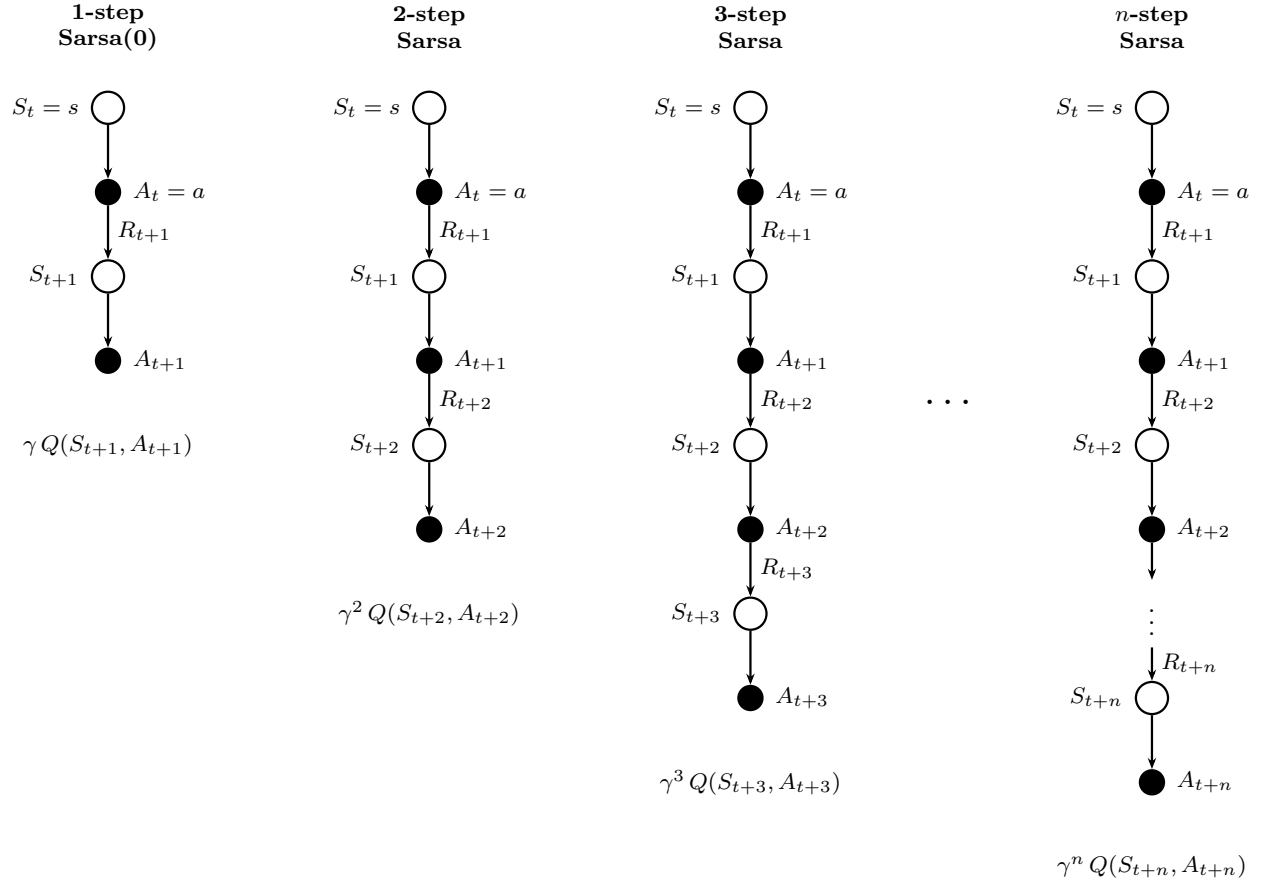


Figure 1: The spectrum of n -step SARSA backups for action values. Each column is one method, drawn as a single trajectory under π from the fixed starting pair $(S_t, A_t) = (s, a)$ at the top. Empty circles are states, filled circles are actions; edge labels are the sampled rewards. The method takes n real steps, accumulating $R_{t+1}, \dots, R_{t+n}$ discounted by $\gamma^0, \dots, \gamma^{n-1}$, then bootstraps with the discounted estimate $\gamma^n Q(S_{t+n}, A_{t+n})$ at the final pair (bottom label). One-step Sarsa(0) bootstraps immediately after one reward; deeper methods push the bootstrap further out; the Monte Carlo limit $n \rightarrow \infty$ reaches termination with no bootstrap.

4 Closing Remarks

4.1 The Lag in Updating

One practical consequence of unrolling is worth stating, because it is visible directly from the n -step target and not from anything specific to implementation. The target $q_t^{(n)}$ for the pair visited at time t depends on the rewards $R_{t+1}, \dots, R_{t+n}$ and on the bootstrap pair (S_{t+n}, A_{t+n}) . None of these exist at time t ; they only become available after n further steps. So the update for time t cannot be applied until time $t + n$. SARSA(0) had no such lag because its target needs only the very next transition. The deeper the unroll, the longer the update waits for its window to fill. At an episode boundary the last few windows never fill completely, and those tail targets simply use the rewards available up to termination with no bootstrap, which is exactly the Monte Carlo target

for that tail.

4.2 Convergence

The convergence story is inherited from SARSA(0) with no new ingredients. Under the same three conditions (every pair visited infinitely often, Robbins-Monro step sizes, and a policy that becomes greedy in the limit while retaining exploration), n -step SARSA converges to q^* for any fixed $n \geq 1$. The one structural fact that does change with n is the contraction modulus: the n -step Bellman operator is a γ^n -contraction, since unrolling n times composes the one-step γ -contraction with itself n times. A deeper unroll contracts faster per update, though each update also costs n environment steps and carries higher variance. The formal stochastic-approximation argument is the same one deferred to the stochastic approximation report.

4.3 Summary

Key Result

n -step SARSA. Unroll the Bellman expectation equation n times before bootstrapping, sample the resulting n -step equation along one trajectory, and step Q toward the n -step target:

$$q_t^{(n)} = \sum_{i=1}^n \gamma^{i-1} R_{t+i} + \gamma^n Q(S_{t+n}, A_{t+n}), \quad Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (q_t^{(n)} - Q(S_t, A_t)).$$

The only new idea relative to SARSA(0) is unrolling: the bootstrap term is itself an action-value, so it satisfies the same Bellman equation, and substituting that equation in for it trades one bootstrap for one more real reward plus a bootstrap one step deeper. The depth n is a single knob interpolating between SARSA(0) ($n = 1$: low variance, high bootstrap bias) and Monte Carlo control ($n \rightarrow \infty$: high variance, no bootstrap bias). The next report keeps every n -step target derived here but stops choosing a single n : it averages all of them with geometric weights, giving SARSA(λ).